Summary

This is a chat moderation bot written in C#. It is contained within a WPF client to provide a graphical user interface for ease of use. It is designed to accept chat messages from one or more platforms, assess them for prohibited phrases or requested actions, and respond to the originating platform as required. This allows the user to centralize their moderation systems across all platforms where they stream.

Background

In 2015, I wrote a simple chat moderation bot for a Twitch streamer. I expanded upon its feature set and user base to over a dozen active streamers, actively supporting it for about a year before it was no longer required. At the end of 2025, I revisited the idea. I redesigned it from scratch and wrote a new chat moderation bot in C# to reflect my current programming and architecture design skills.

Feature Set

- Message monitoring in real time for multiple platforms simultaneously.
- Automatic moderation actions based on user-defined filter rules. Supports plain text as well as regular expression filters.
- Custom command configuration along with several core functionality commands for controlling filter status, command registry, and user permissions.
- UI supports both English and Japanese. Users are able to switch their chosen language at any time.

How to Use

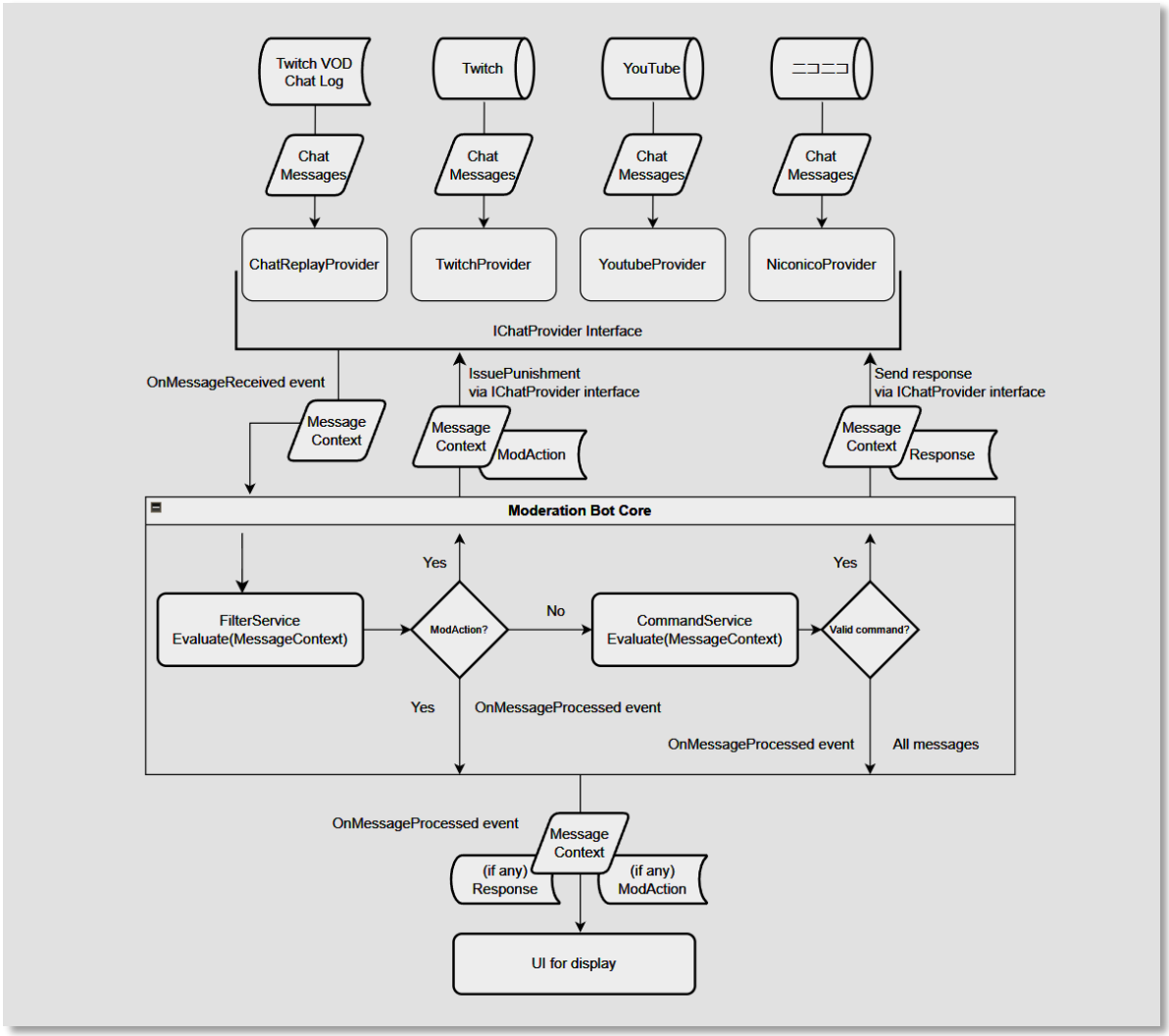
1. Launch 'Chat Moderation Bot.exe' from the program folder.
2. From the File menu, select Start New Provider.
3. Choose the Twitch VOD Chat Replay option and select a message density.
4. Enter a name in the Platform Identity field.
5. Press OK. A provider window will open and begin playback. Messages can be entered using the text input field at the bottom of the window.

Technical Overview

The bot is implemented in C# with a UI client based on the WPF framework. Supported platforms have a dedicated chat provider module which handles platform connections. This provider implements the IChatProvider interface, which enforces standards on incoming chat messages to ensure smooth processing and routing. It also ensures that the bot is able to communicate with each platform as needed.

The providers convert incoming chat messages into MessageContext objects. The bot accepts these messages using event-based notifications. The MessageContext is then passed through the Filter and Command services, which decide whether a reaction is necessary. Any necessary reactions are sent back to the original provider using the IChatProvider interface. The processed message is sent to the UI for display using event-based notifications.

The diagram below demonstrates the full processing pipeline.



Once the core functionality was in place, unit testing was used to ensure edge cases were covered and all assumptions made during planning and development lined up with reality.

A Note on Japanese Language Support

The application UI supports both English and Japanese. The default language is Japanese and users

may switch at any time by selecting the 言語/Language option in the menu.

Chat message processing itself is language-agnostic. However, the default settings and provided chat logs assume an English-language chat environment.

Japanese language parsing involves significantly different linguistic processing requirements including complex normalization and tokenization processes that are outside of the scope of this project. However, the system is designed to allow for such extensions in the future with minimal restructuring.

Retrospective

My original vision for this project was to revitalize the chat moderation bot I'd written previously with the skills I've developed since that time. I had three main goals for this:

1. Revise the structure of the program to be more flexible and modular.
2. Allow for one bot to manage multiple platforms simultaneously.
3. Make the user experience more palatable to others.

The first goal was achieved by using a more robust planning period, carefully considering the program's assumptions and needs. This also allowed me to design the chat provider system so that the second goal could be achieved at the same time.

The third goal required building a graphical UI to contain the bot instead of relying on the console terminal. While the resulting UI is simple, it is functional and cleaner than relying purely on console terminal. It also provides a solid foundation upon which to expand functionality for future development. However, the time it took to learn what I needed about WPF and implement it was longer than anticipated.

There are some remaining aspects of the program with which I am displeased. During development, I toyed with the idea of having the provider window include additional windows showing all currently active commands, filters, and permissions. However, because this was not considered during the planning phase, retrofitting the project to support this would have required a significant time investment. Performance is also a concern; because there is no culling system for the provider window, UI performance degrades after a large number of messages have been displayed. This would be my first priority were I to continue development of this project.

Overall, I am pleased with the results.

Authorship

All code written by the author. Platform logos retrieved from each platform's public branding package for this purpose.